Data Structure Using C++
Lecture : 2

Static and Dynamic Allocation

(ADT & Arrays)

Dr. Ali Hamzah

Arrays Review

- A collection of identically typed data items distinguished by their indices (or subscripts)
- One dimensional arrays

  type identifier[size];

  lower_bound = 0, upper bound = size –1

```
int ar[100];
```

Arrays Review

- **Initializing Arrays**

  type name [elements];

  int x[5];

  int x[3]={1,2,3};

  int x[10]={1};

must be constant

Arrays Review

float a[3] = { 0.1, 0.2, 0.0 };

int a[3] = { 3, 4 };

int a[] = { 2, -4, 1 };≡ int a[3] = { 2, -4, 1 };

char s[] = “abcd”; ≡ char s[] = { ‘a’,‘b’,‘c’,‘d’,‘\0’ }

- **Accessing Arrays**

  array[index] ----------> cout<< x[2]; x[1]=100;

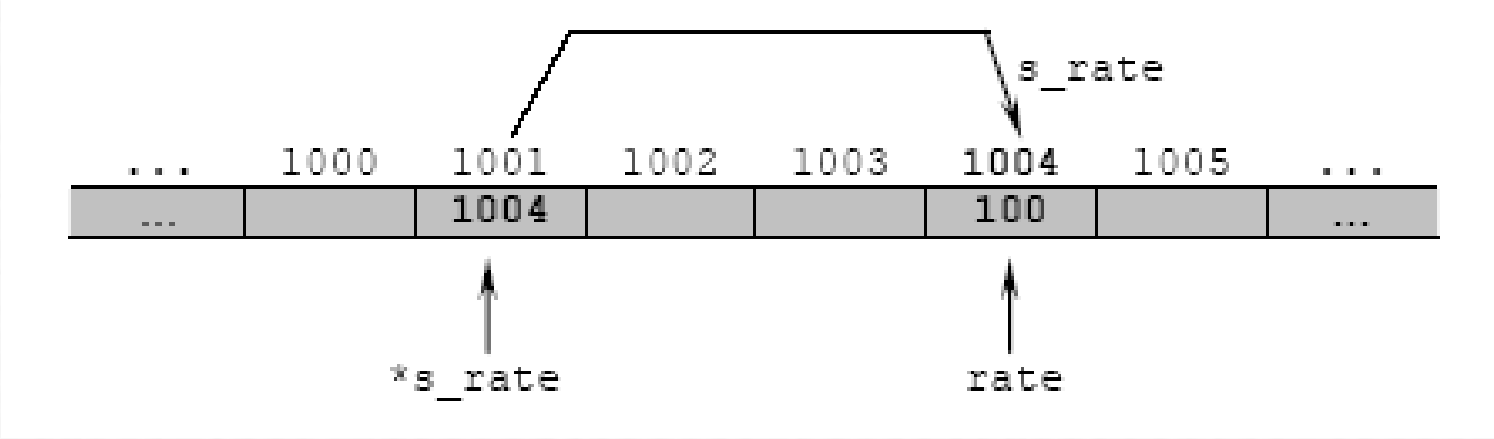
Pointers revisions

Point to here,
point to there,
point to that,
point to this,
and point to nothing!
well, they are just memory addresses!!

Pointers

int *s_rate;

s_rate is pointing to rate or is said a pointer to rate



Pointers

s_rate is pointing to rate or is said a pointer to rate

Where:

address	variable name	hold data

Pointers(operators)

1- Indirection operator (*) –
Declaring → int* x;
Dereferencing → *x= 50;

2- Address-of-operator (&) – Return the address of.

int x; int* y; y=&x;

Pointers(operators)

```
int main() {
    int value = 10;
    int* ptr = &value;  // Pointer variable "ptr" stores the memory address of "value"

    *ptr = 50;  // Dereferencing ptr and assigning the value 50 to the memory location it points to

    cout << ("Value", value);  // Output: Value: 50

    return 0;
}
```

Introduction to Data Structures

- **What is Data ?**
  - **Any useful information – that can be stored or memorized for future reference**
    - **In an organization it can be**
      - **Employee's name, sex, age, department, address so on**
    - **In Railway reservation office**
      - **Passenger name, traveling date, traveling time, seat number so on**
    - **In Computers**
      - **Set of integers, Structure(s) or any other user defined data type**

Static And Dynamic Allocation

Stack allocation **Static Array**

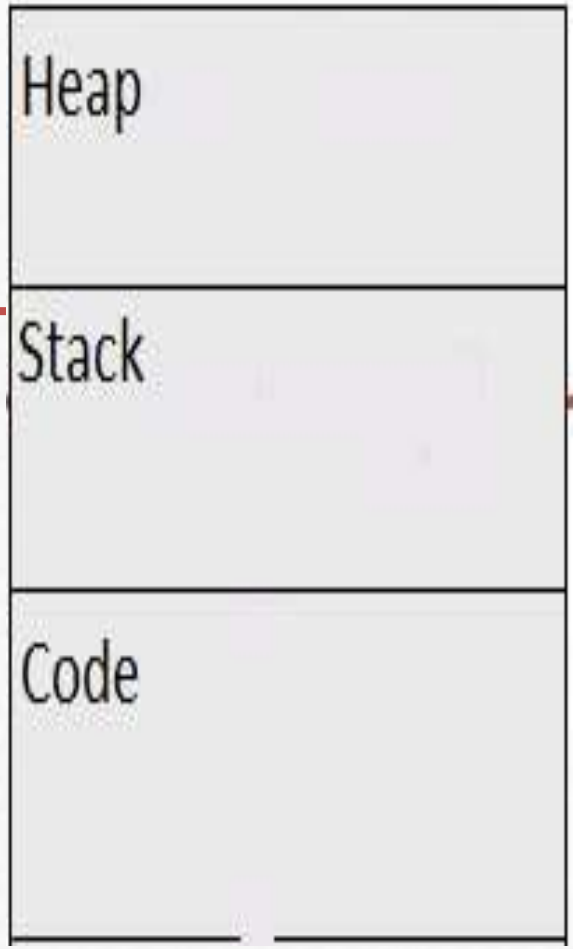
```
int intArray[10];
intArray[0] = 6837;
```

Heap allocation **Dynamic Array**

```
int *intArray;
intArray = new int[10];
intArray[0] = 6837;

...

delete[] intArray;
```

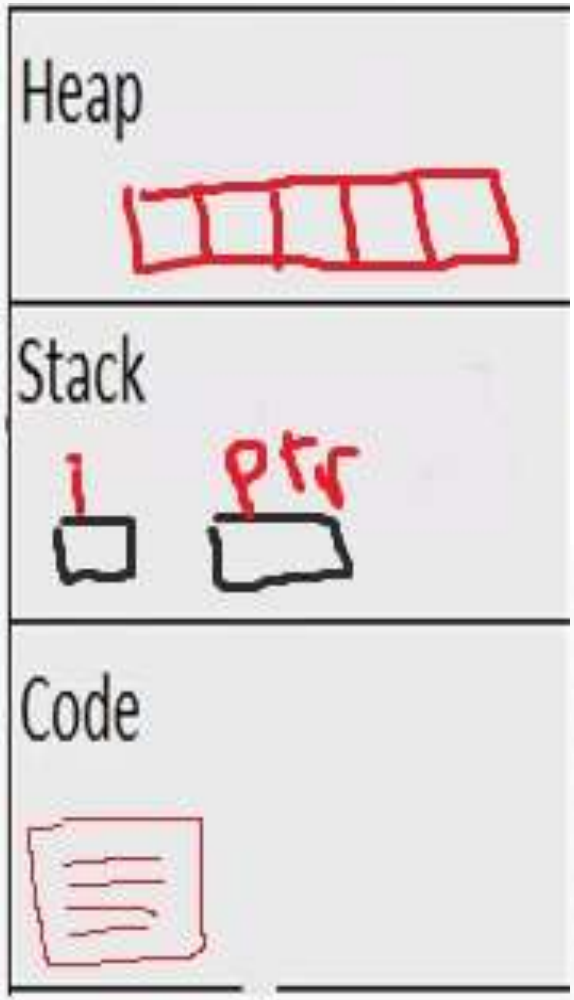


Static And Dynamic Allocation (cout'd)

Heap allocation Dynamic Array

```
int main()
{

    int i ;
    float * ptr;
    cout<<"Enter array size \n";
    cin>>i;
     ptr = new float[i];
    delete [] ptr;        // Delete array
    return 0;
}
```



Static & Dynamic Arrays

- **Static Array: has a fixed number of elements and it's allocated during compilation time;**

  stack

  Ex: int x[20];

- **Dynamic Array: an array whose size is determined when the program is running, not when you write the program**

  head

- **created using allocation techniques and dynamic memory management. Can be resized.**

  Ex: int *x = new int [20];

Allocation- Dynamic Array

- **Allocate an array with "new"**

```
int* a = NULL; // Pointer to int, initialize to nothing
 int n; // Size needed for array
 cin >> n; // Read in the size
a = new int[n]; // Allocate n ints and save ptr in a
 for (int i=0; i<n; i++) {
a[i] = 0; // Initialize all elements to zero.
} . . . // Use a as a normal array
delete [] a; // When done, free memory pointed to by a.
a = NULL; // Clear a to prevent using invalid memory reference.
```

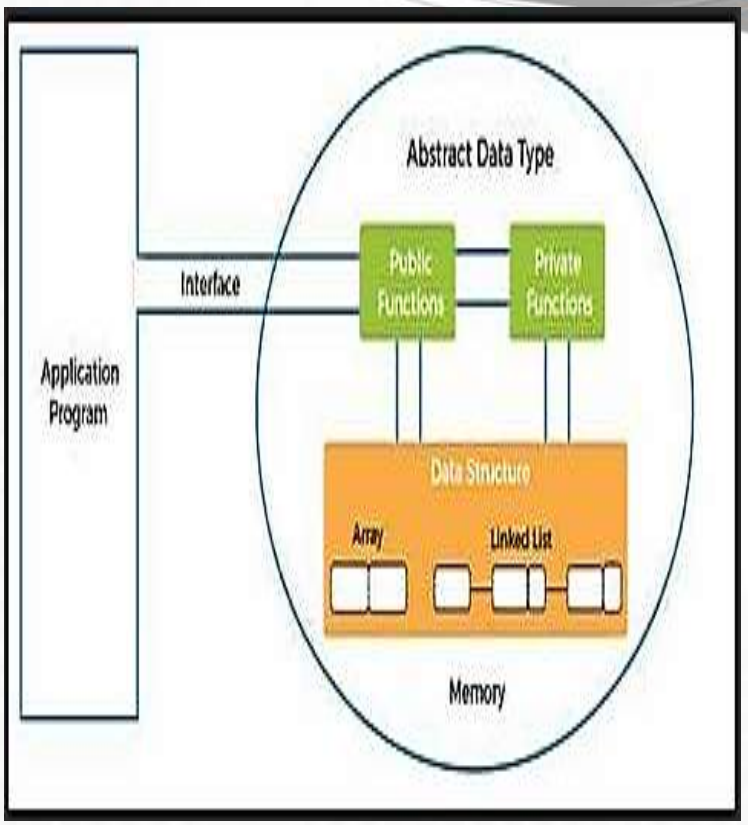
Abstract Data Types (ADT)

- **What is Data Structure?**
  special
  - Particular way of storing and organizing data in a computer so it can be used efficiently
  - It group od data elements stored together under one name
  - ***Also referred as Abstract data type (ATD)***
- **For what purpose?**
  - **To facilitate efficient**
    - » **Storage of data**
    - modify » **Manipulate of data**
    - display » **Retrieval of data**

Abstract data type (ATD)

- **An abstract data type (ADT)** is a high-level description of a data structure that specifies the operations that can be performed on the data without specifying the implementation details.

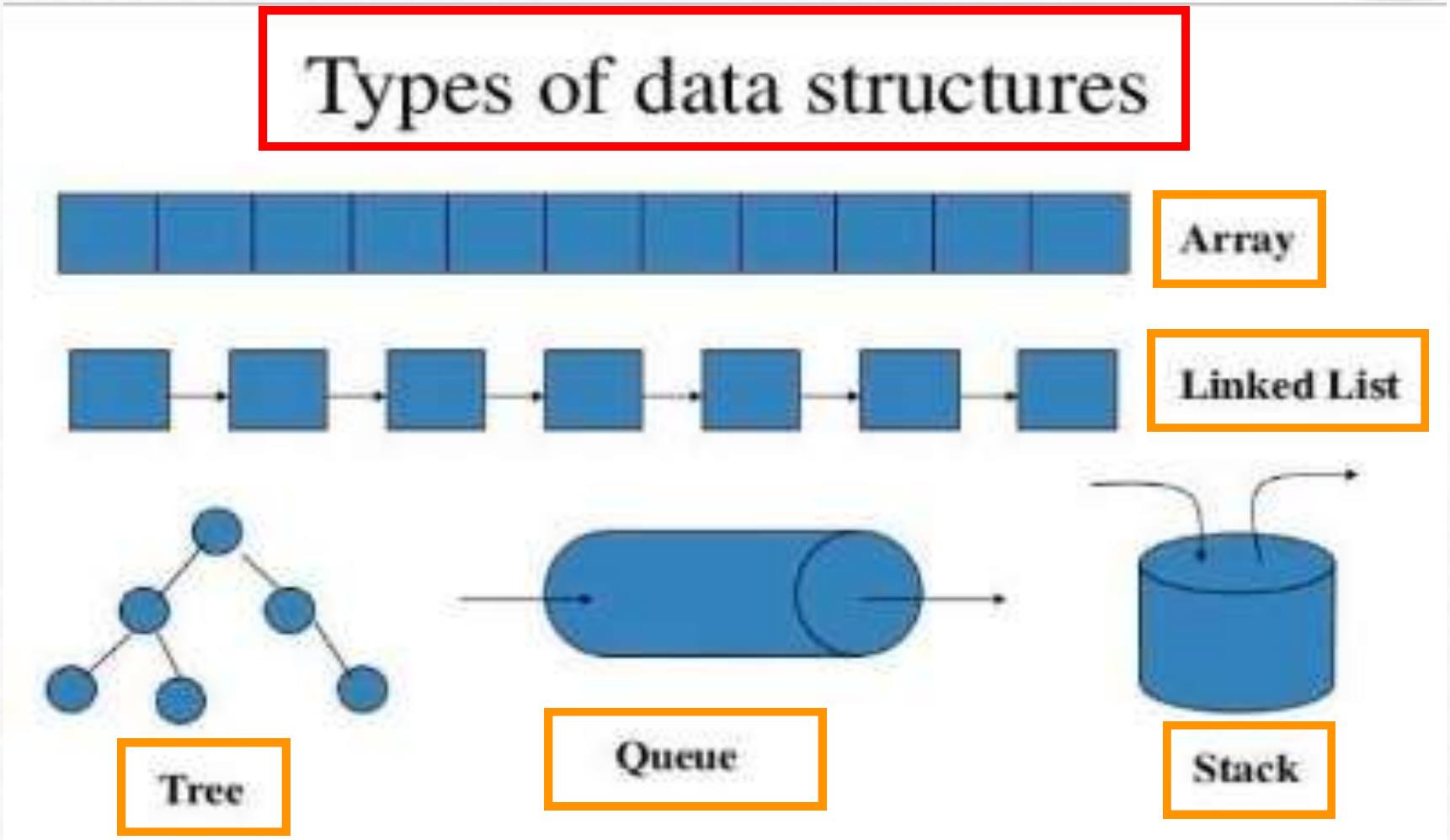
- **It defines** a set of functions or methods that can be used to manipulate the data, while hiding the internal representation and organization of the data



Data Structures ADT

- **Collection of data in an ordered fashion**
  - Ages of the students in a class are all numbers (data), but once are grouped in one line, one after the other becomes **structured data** (arrays)
  - Age, class, sex, height, weight of a student is all data, but if we place them in one line per student it would be structured data (structures/struct)
  - Arrays, list, queues, stack, doubly linked lists, structures etc are all data structures
- **Different operations** are defined for each type of data structure to add, delete or modify its content

Types of Data Structures



Classification of data structures

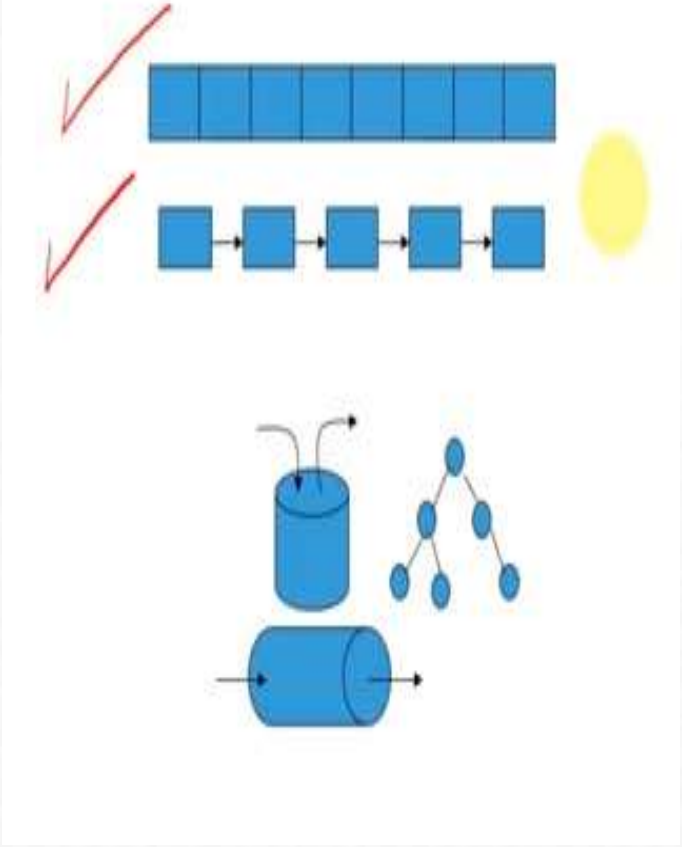
➢ Based Existence

❖ Physical data structures

- Array
- Linked List

❖ Logical data structures

- Can’t be created independently
- Stack, queues, trees, graph

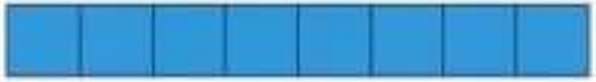


Aspect	Logical Data Structures	Physical Data Structures
Representation	Abstract, conceptual organization of data.	Actual storage format in memory or on storage devices.
Examples	Arrays, linked lists, stacks, queues, trees, graphs.	Arrays with contiguous memory, linked lists with scattered blocks, B-trees, hash tables.
Focus	Relationships and operations on data.	Efficiency considerations, storage details.

Classification of data structures

➢ Based on Memory Allocation

- ❖ **Static(or fixed sized) data structure**
  - Such as Array



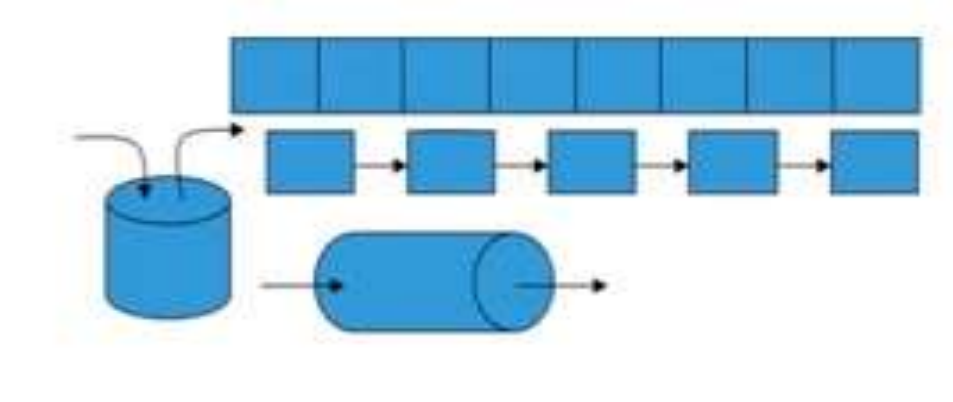
- ❖ **Dynamic data structure (change size as needed)**
  - Such as linked list

Classification of data structures

➢Based on Representation

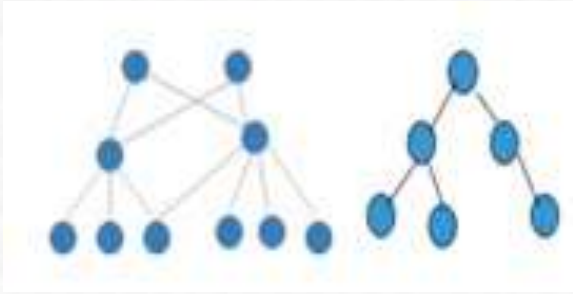
❖Linear Data structures

- Array
- Linked list
- Stack
- queue



❖Non Linear Data structure

- Trees
- Graph



Operation of data structure

- **Traversing**
  - Accessing/vesting each data elements exactly once so the certain elements in the data may be processed
- **Searching**
  - Finding the location of the given data elements(key)
- **Insertion**
  - Adding a new element to the structure

Operation of data structure

- **Deletion**
  - Removing element data from the structure
- **Sorting**
  - Arrange the data elements in some logical fashion
    - **Ascending/descending**
- **Merging**
  - Combine data elements from two or more data structure into one

Algorithms

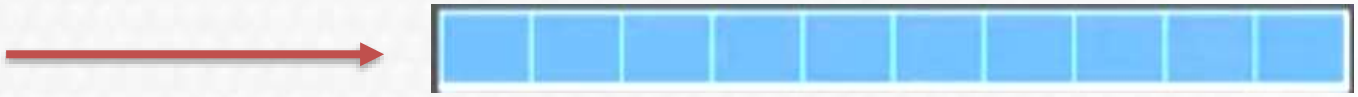
- The operations defined on the data structures are generally knows as algorithms
- For each data structure there has to be defined algorithms
- Algorithms are normally used to add, delete, modify, sort items of a data structure
- All programming languages are generally equipped with conventional data structures and algorithms.
- New data structures and the algorithms for manipulation can be defined by the user

Design Issue

» ***The challenge is to select the most appropriate data structure for the problem***

Arrays As ADT

Arrays ADT

- **Data**
  - **Sequential collection of elements of the same type**
  - **A collection of variables of the same type.**
- **Operations :**
  - ***Create ()*** 
  - **Fill**
  - **Display()**
  - **Append (int newitem)**
  - **Insert (int index, int newitem)**
  - **Search (int key)**

Arrays ADT

- Data
  - Sequential collection of elements of the same type
  - A collection of variables of the same type.
- Operations :
  - Create ()
  - *Fill*

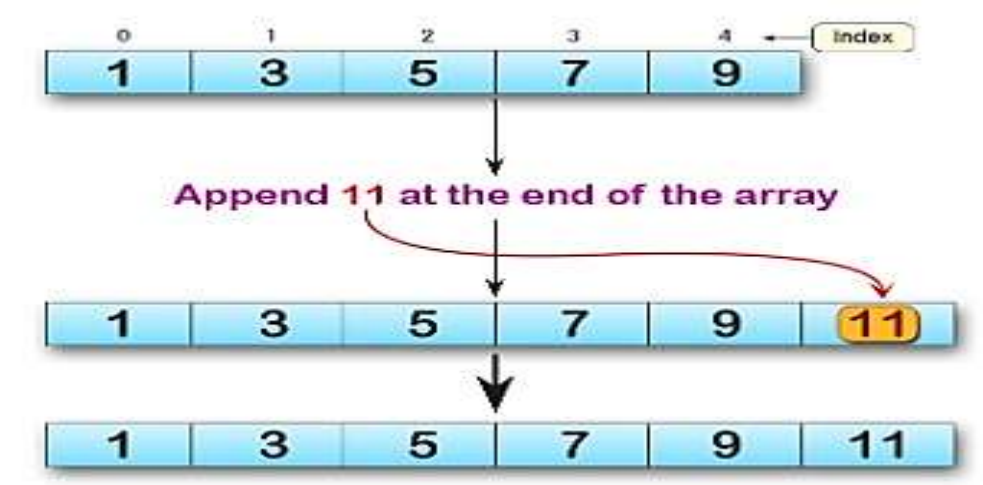

```
void fillArray (int value) {
    for (int i = 0; i < array.size(); i++) {
        array[i] = value;
    }
```

Arrays ADT

- **Operations :**
  - **Display()**

```
void printArray() {
    for (int i = 0; i < array.size(); i++) {
      cout << array[i] << " ";
      }
```

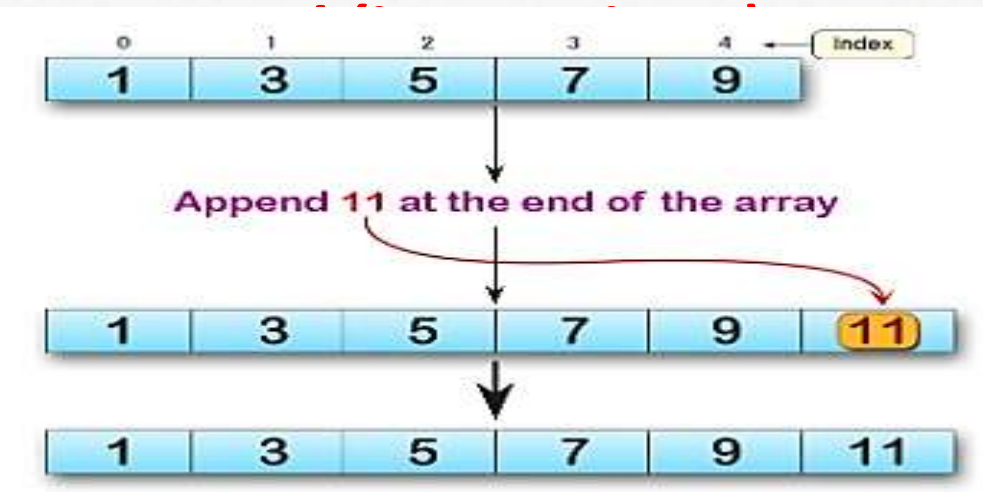
  - ***Append (int newitem)***



Arrays ADT

- **Operations :**
  - **Search(int key)**

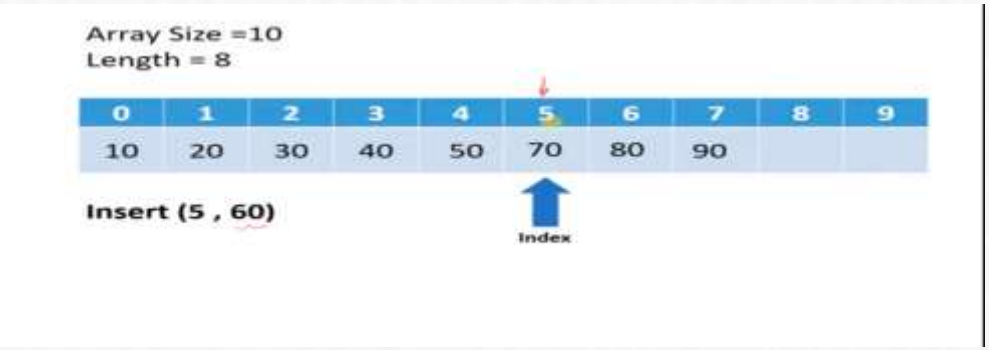
```
void search(int k) {
    for (int i = 0; i < array.size(); i++) {
      if ( array[i]== key);
      cout << array[i] << " ";
      }
```



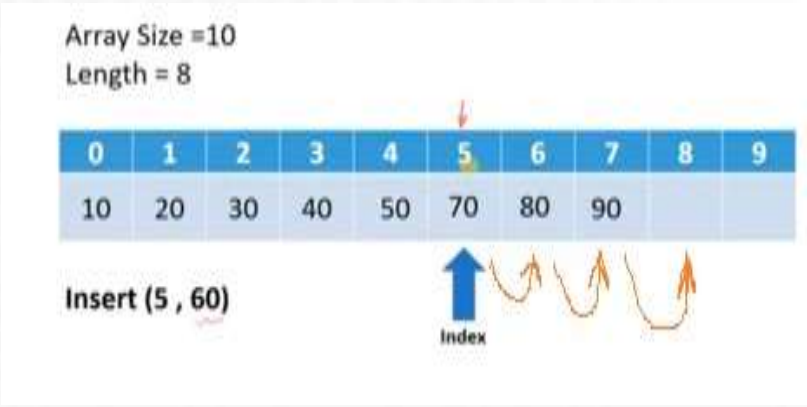
Arrays ADT

- **Operations :** ***Insert (int index, int newitem)***

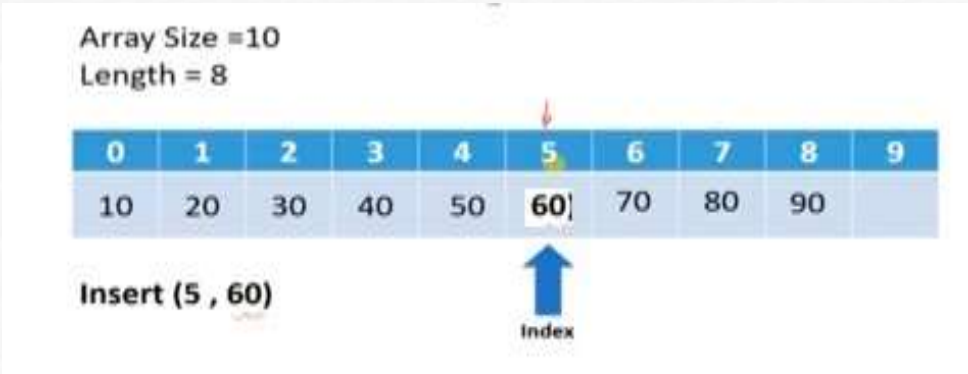
1



2



3



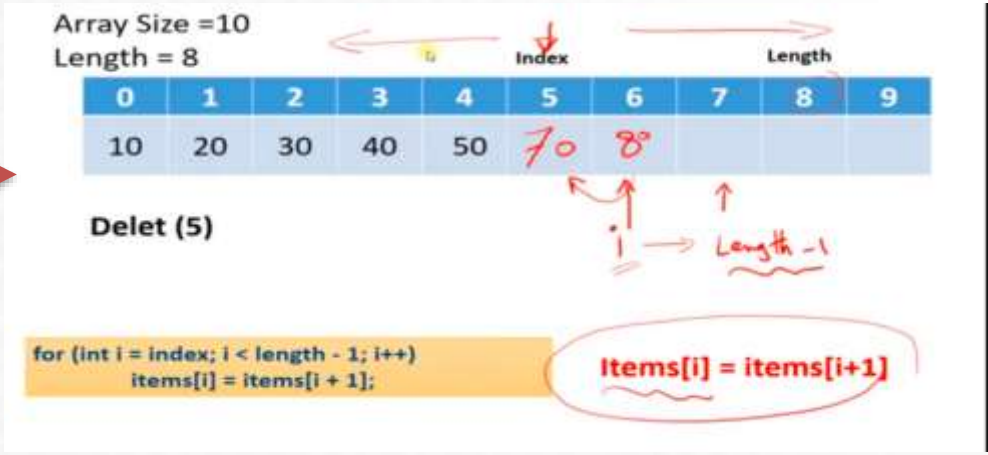
Arrays ADT

- **Operations :** ***Insert (int index, int newitem)***

```
Enter no. of elements you want for the array:
5
Enter 5 elements for the array:
23
65
36
14
52
Enter the element you want to insert:
65
Enter position for the newitem element:
3
Array after inserting newitem element
 23 65 65 36 14 52
--------------------------------
Process exited after 20.93 seconds with return value 0
Press any key to continue . . .
```

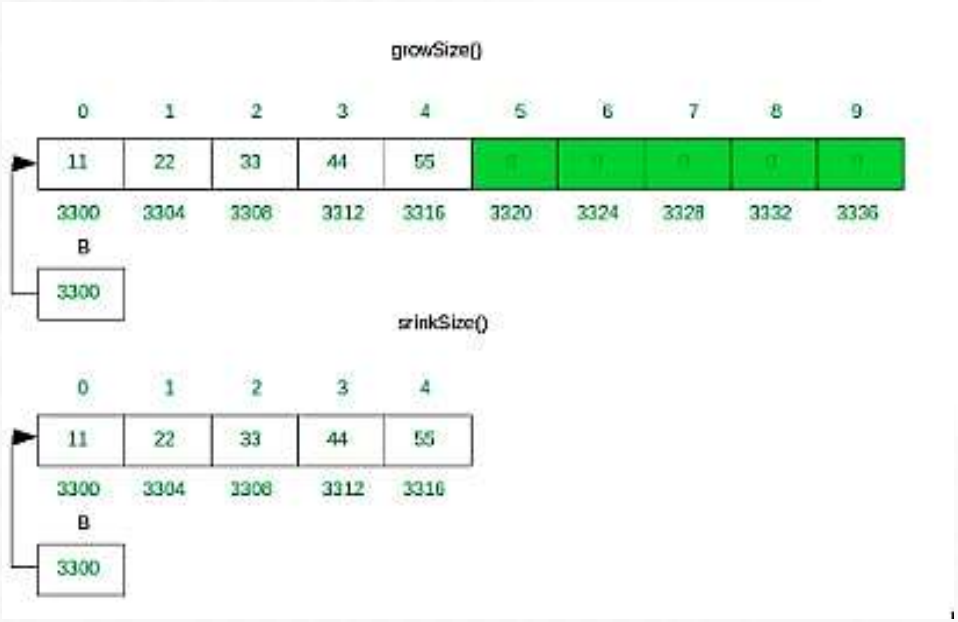
Arrays ADT

- **Operations :**
  - ***Delete (int index)***
  - **Enlarge (newSize)**
  - **Merge (Array other)**
  - **getSize ()**
  - **getLength ()**



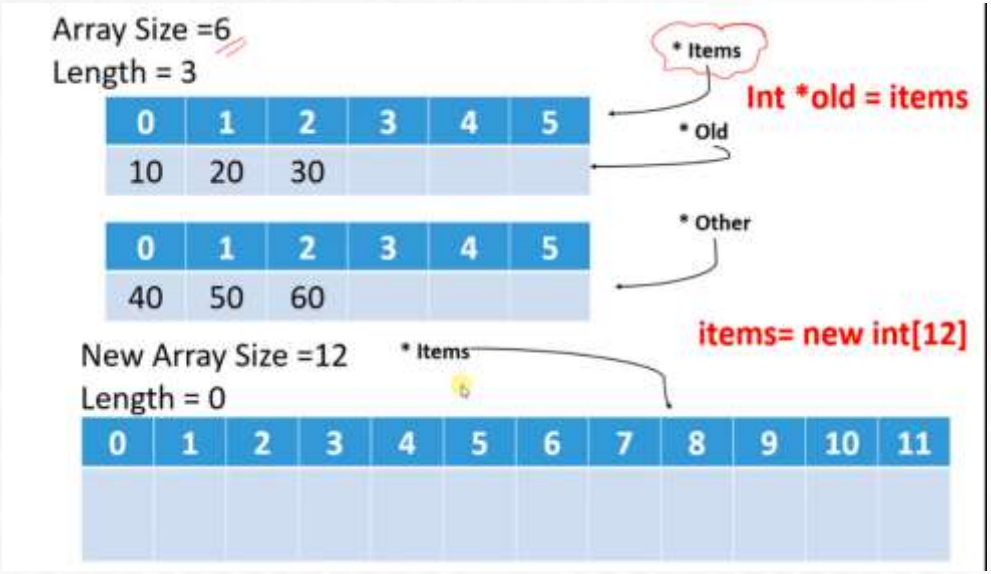
Arrays ADT

- **Operations :**
  - ***Enlarge (newSize)***
  - **Merge (Array other)**
  - **getSize ()**
  - **getLength ()**



Arrays ADT

- **Operations :**
  - ***Merge (Array other)***
  - **getSize ()**
  - **getLength ()**



Problem with Arrays

- Normal arrays require that the programmer determine the size of the array when the program is written
  - What if the programmer estimates too large?
    - Memory is wasted
  - What if the programmer estimates too small?
    - The program may not work in some situations
- Dynamic arrays can be created with just the right size while the program is running

Multidimensional Dynamic Arrays

- To create a 3x4 multidimensional dynamic array
  - View multidimensional arrays as arrays of arrays
  - First create a one-dimensional dynamic array
    - create a dynamic array of pointers named m:

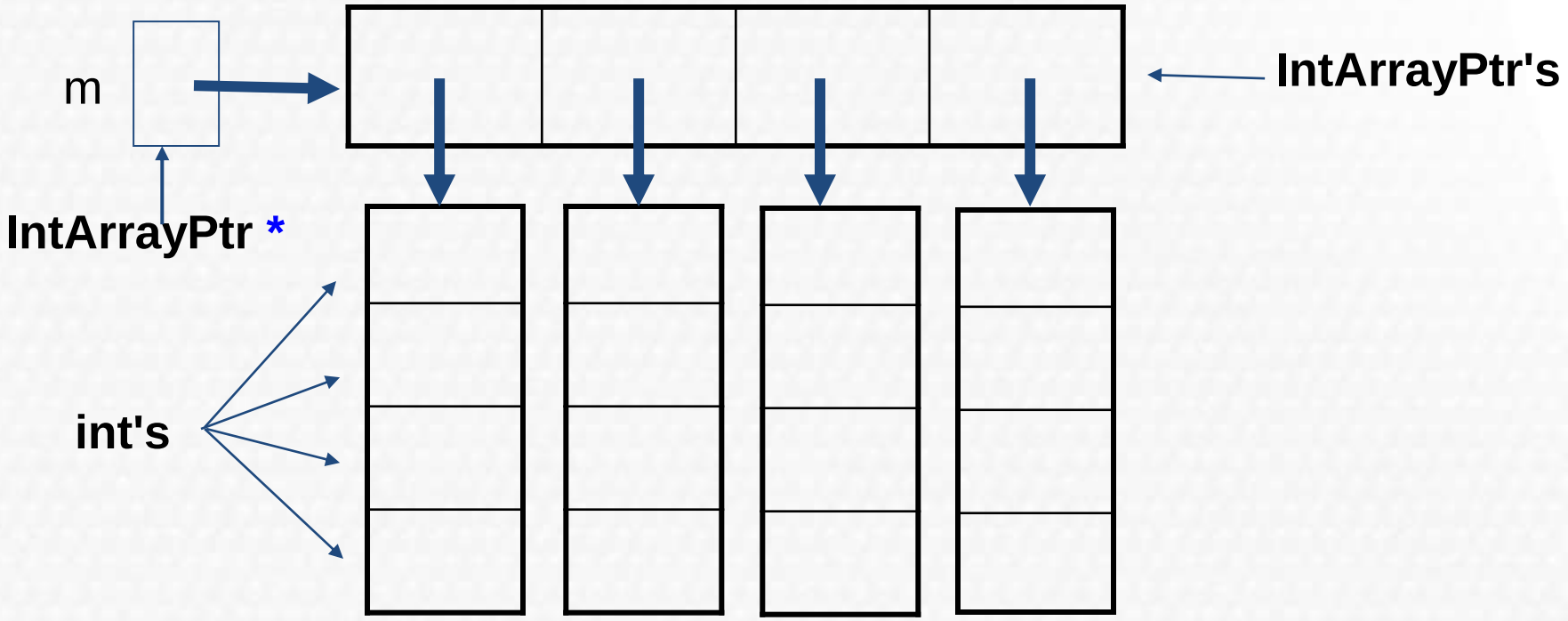
```
IntArrayPtr *m = new IntArrayPtr[3];
```

  - For each pointer in m, create a dynamic array of ints
    -

```
for (int i = 0; i<3; i++)
    m[i] = new int[4];
```

Multidimensional Dynamic Arrays

- The dynamic array created on the previous slide could be visualized like this:



Deleting Multidimensional Arrays

- **To delete a multidimensional dynamic array**
  - Each call to new that created an array must have a corresponding call to delete[ ]
  - Example: To delete the dynamic array created on a previous slide:

```
for ( i = 0; i < 3; i++)
 delete [ ] m[i]; //delete the arrays of 4 int's
delete [ ] m; // delete the array of IntArrayPtr's
```

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Rows = 4
    // Cols = 6
    int **ptr = new int *[4]; // Declare a pointer to an array of integers

    for (int i = 0; i < 4; i++)
    {
        ptr[i] = new int[6]; // Assign each element of ptr to a new array of integers
    }

    // Your code here

    // Don't forget to free the allocated memory when done
    for (int i = 0; i < 4; i++)
    {
        delete[] ptr[i]; // Delete each array
    }
    delete[] ptr; // Delete the array of pointers

    return 0;
}
```